HTTP 和 HTTPS 的主要区别如下：

1. 安全性

特性	HTTP	HTTPS
加密	❌ 明文传输	✅ SSL/TLS 加密
数据安全	容易被窃听、篡改	数据加密，防止中间人攻击
身份验证	无	通过 CA 证书验证服务器身份

2. 端口号

- **HTTP**: 默认端口 `80`
- **HTTPS**: 默认端口 `443`

3. 证书

- **HTTP**: 不需要证书
- **HTTPS**: 需要 SSL/TLS 证书（可从 CA 机构获取，或使用 Let's Encrypt 免费证书）

4. 性能

- **HTTP**: 连接建立较快（TCP 三次握手）
- **HTTPS**: 稍慢（TCP 握手 + TLS 握手），但现代优化（如 TLS 1.3、HTTP/2）已大幅减少延迟

5. URL 前缀

```
http://example.com
https://example.com

```

6. 浏览器表现

- **HTTP**: 现代浏览器会标记为"不安全"
- **HTTPS**: 显示🔒 锁图标，表示安全连接

HTTPS 工作原理简述

```
客户端                                服务器
   |                                    |
   |------ 1. 请求建立连接 ------->|
   |<----- 2. 返回 SSL 证书 ------|
   |------ 3. 验证证书有效性 ----->|
   |<---- 4. 交换加密密钥 --------|
   |====== 5. 加密通信开始 =======|

```

SSL/TLS 采用**混合加密**机制，结合了对称加密和非对称加密的优点：

加密方式概览

```
┌──────────────────────────────────────────────────────────────┐
|                        TLS 加密体系                          |
├──────────────────────────────────────────────────────────────┤
|  非对称加密（慢但安全）    →  用于密钥交换、身份验证            |
|  对称加密（快速高效）      →  用于实际数据传输                  |
|  哈希算法                  →  用于数据完整性校验                 |
└──────────────────────────────────────────────────────────────┘

```

---

1. 非对称加密（Asymmetric Encryption）

用于 **TLS 握手阶段**，交换对称密钥。

算法	密钥长度	特点
RSA	2048/4096 位	经典算法，用于密钥交换和签名
ECDHE	256/384 位	椭圆曲线，更短密钥实现同等安全性
ECDSA	256/384 位	椭圆曲线签名算法

```
公钥加密，私钥解密：
```

```

客户端                                    服务器
    |                                        |
    |<------ 发送公钥（证书中）---------|
    |                                        |
    |-- 用公钥加密"预主密钥"发送 ------>|
    |                                        |
    |        服务器用私钥解密，获得预主密钥   |

```

---

2. 对称加密（Symmetric Encryption）

用于 **实际数据传输**，速度快。

算法	密钥长度	特点
AES-GCM	128/256 位	主流选择，带认证的加密模式
AES-CBC	128/256 位	较旧，需配合 HMAC
ChaCha20-Poly1305	256 位	移动端友好，无需硬件加速

```
同一密钥加密解密：

明文 "Hello"
     ↓ 密钥加密
密文 "x7Fk9..."
     ↓ 同一密钥解密
明文 "Hello"

```

---

3. 哈希/消息认证（Hash/MAC）

确保数据**完整性**，防止篡改。

算法	输出长度	用途
SHA-256	256 位	证书签名、密钥派生
SHA-384	384 位	更高安全级别
HMAC	可变	消息认证码

TLS 1.3 握手流程（简化）

```
客户端                                          服务器
   |                                               |
   |-------- ClientHello -------------------->  |
   |         (支持的加密套件、随机数)              |
   |                                               |
   |<------- ServerHello --------------------- |
   |         (选定的加密套件、证书、随机数)        |
   |                                               |
   |-------- 密钥交换参数 -------------------->  |
   |                                               |
   |  ======== 双方计算出相同的会话密钥 ========   |
   |                                               |
   |<======= 加密通信开始 ==================>  |

```

密码套件示例

TLS 使用**密码套件（Cipher Suite）** 组合各种算法：

```
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
 |    |     |        |    |     |
 |    |     |        |    |     └── 哈希算法: SHA384
 |    |     |        |    └──────── 加密模式: GCM
 |    |     |        └─────────────── 对称加密: AES-256
 |    |     └──────────────────────── 身份验证: RSA
 |    └────────────────────────────── 密钥交换: ECDHE
```

```
└──────────────────────────────────── 协议：TLS

```

TLS 1.3 推荐套件

```
TLS_AES_256_GCM_SHA384          ✅ 高安全性
TLS_AES_128_GCM_SHA256          ✅ 平衡性能和安全
TLS_CHACHA20_POLY1305_SHA256    ✅ 移动设备优选

```

为什么用混合加密？

加密类型	优点	缺点
非对称加密	安全交换密钥，无需预共享	速度慢（约慢 1000 倍）
对称加密	速度快，适合大量数据	密钥分发困难
混合使用	取长补短	✅ 最佳方案

总结

```
┌──────────────────────────────────────────────┐
│                  TLS 安全通信                  │
├──────────────────────────────────────────────┤
│  🔐 机密性：AES/ChaCha20 对称加密              │
│  🖋️ 完整性：SHA-256/HMAC 哈希校验             │
│  ✅ 身份性：RSA/ECDSA 证书验证                 │
│  🔑 密钥交换：ECDHE 前向保密                   │
└──────────────────────────────────────────────┘

```

- **某些浏览器接口需要https环境下才能使用**

- Clipboard
- Geolocation